Міністерство освіти і науки України
Вінницький національний технічний університет
Кафедра програмного забезпечення

КУРСОВА РОБОТА

з дисципліни ” Алгоритми та структури даних ”

на тему: «Розробка й аналіз ефективності алгоритмів і програм»

студента __2__ курсу_групи 2ПІ-25Б програмного забезпечення спеціальності 121 “Інженерія забезпечення” Коваля О.Д. _____

Керівник: доцент кафедри ПЗ, к.т.н., _____ Ткаченко О.М
(підпис)

Кількість балів _____ Оцінка ECTS_____

м. Вінниця – 2025 рік

Факультет інформаційних технологій та комп’ютерної інженерії

Кафедра програмного

забезпечення

ЗАТВЕРДЖУЮ

Зав. кафедри ПЗ, проф., д.т.н.

_____ О.Н. Романюк

(підпис)

” 11 ” вересня 2025 р.

ІНДИВІДУАЛЬНЕ ЗАВДАННЯ

на курсову роботу з дисципліни “Алгоритми та структури даних”

Студенту Ковалю Олександру Дмитровичу

факультету *ІТКІ* групи 2ПІ-25Б

1. Завдання №74. Сокобан (Sokoban).

На ігровому полі розміром M×N, що містить довільну конфігурацію стін та перешкод, розташовано комірника, K ящиків та K цільових комірок (складів). За один хід комірник може переміщуватися на сусідню вільну клітинку по горизонталі або вертикалі, а також штовхати перед собою рівно один ящик, якщо клітинка безпосередньо за ящиком є вільною. Тягнути ящики назад або переміщувати більше одного ящика одночасно заборонено. Мета полягає в тому, щоб розташувати всі K ящиків на цільові комірки за мінімальну кількість переміщень або штовхань. Програма повинна забезпечувати як ручний режим проходження гри користувачем, так і автоматичний пошук та покрокову візуалізацію знайденого розв’язку за допомогою різних алгоритмів пошуку на графах станів. Передбачити виявлення «тупикових» (deadlock) станів, коли ящик потрапляє в кут чи глухий кут, звідки його неможливо вивести. Можливість інтегрувати ШІ для вирішення завдання, реалізація щонайменше 2 алгоритмів для вирішення.

2. Постановка задачі

2.1. Обрати або розробити метод розв’язку завдання.

2.2. Розробити щонайменше два алгоритми розв’язку завдання для пошуку шляху в просторі станів: алгоритм пошуку в ширину (BFS) для знаходження оптимального розв’язку на невеликих рівнях із мінімальною кількістю

переміщень; евристичний алгоритм A* (або IDA*) з евристикою на базі суми манхеттенських відстаней від ящиків до найближчих цілей (або задачі про призначення Hungarian algorithm) із відсіканням тупикових конфігурацій (Deadlock Detection). Оцінити витрати часу та пам’яті, необхідні для реалізації розроблених алгоритмів.

2.3. Розробити необхідні класи та об’єкти (ігрове поле, стан системи, ящик, комірник, вузол дерева пошуку). Розробити функції та змінні класів.

2.4. Використовуючи технологію об’єктно-орієнтованого програмування, розробити програму для виконання завдання. У програмі мають використовуватися щонайменше дві з наведених нижче структур даних: масиви, стеки, деки, черги, списки, дерева, хеш-таблиці (наприклад, пріоритетна черга для відкритих станів та хеш-таблиця з хешуванням Зобріста для множини відвіданих станів Closed Set).

2.5. Програма має отримати дані з текстового файлу або графічного редактора (конфігурація рівня з символами стін, цілей, ящиків та гравця у стандартному форматі Sokoban), за допомогою об’єктно-орієнтованої технології реалізувати розроблений метод виконання завдання та вивести результати у графічному режимі на екран.

2.6. Програма повинна обчислювати та виводити на екран час роботи для кожного реалізованого алгоритму (чистий час пошуку розв’язку та час виконання одного ходу при анімації).

2.7. Програма повинна мати зручний для роботи інтерфейс із підтримкою візуалізації, покрокового відтворення та скасування ходів (Undo).

3. Вхідні дані

3.1 Розміри ігрового поля M×N.

3.2 Початкова карта рівня, задана у текстовому файлі (координати стін, комірника, ящиків та цілей) або згенерована випадково/обрана зі списку рівнів.

3.3 Кількість ящиків K та відповідних цільових позицій.

3.4 Вибір режиму роботи (гра користувача або авторозв’язання) та вибір алгоритму пошуку розв’язку (BFS чи A*).

4. Вихідні дані

4.1 Графічне відображення ігрового поля, стану розміщення ящиків та анімація послідовності переміщень комірника і штовхань.

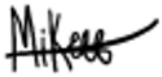
4.2 Знайдена послідовність ходів (наприклад, напрямки U, D, L, R), загальна кількість кроків комірника та кількість виконаних штовхань ящиків (pushes).

4.3 Кількість досліджених станів (вузлів графа) та чистий час роботи кожного з алгоритмів. Час роботи програми включає: час завантаження рівня, час preprocessing (підготовка статичних даних), чистий час пошуку розв'язку, час реконструкції шляху, час валідації рішення та загальний час розв'язання. Час виконання одного ходу при анімації вимірюється окремо.

Дата видачі "__8__" вересня 2025р.

Керівник:

доцент кафедри ПЗ Ткаченко О. М. _____

(підпис) 

Завдання отримав _____ Коваль О.Д.

(підпис) (ПІБ)

АНОТАЦІЯ

Для даної курсової роботи були розроблені алгоритми пошуку розв'язку гри «Сокобан» у просторі станів: алгоритм пошуку в ширину (BFS) та евристичний алгоритм A* з макроштовханнями, зворотними push-відстанями, мінімальним призначенням ящиків цілям (Угорський алгоритм) та безпечним виявленням тупикових станів (Deadlock Detection). Алгоритми є повними, тому завжди знаходять розв'язок, якщо він існує, а A* додатково гарантує оптимальність за обраною метрикою (кількість ходів або штовхань). Окрім базових BFS та A*, ядро містить IDA*, жадібний пошук, навчальний прототип X→Y та популяційні методи ACO, GA, Drain і Cannibal GA; браузерна версія додатково надає біо-агента FlyBrain і зовнішній ШІ (Groq/Google), а всі дванадцять алгоритмів в інтерфейсі пронумеровано від 1 до 12.

Програмна реалізація завдання була створена на мові програмування C++20, що забезпечує високу продуктивність пошуку та можливість повторного використання ядра в різних інтерфейсах. Термінальний інтерфейс реалізовано як консольний застосунок з ANSI-візуалізацією, браузерна версія – на Next.js/React із нативним процесом-мостом sokoban_web, що виконує те саме C++-ядро через JSON-протокол. Проект збирається за допомогою CMake, використано середовище розробки Visual Studio Code та компілятор MinGW-w64.

У результаті виконання курсової роботи було отримано цінні знання з алгоритмізації, оцінювання складності алгоритмів, прийомів програмування. Покращено навички з об'єктно-орієнтованого програмування та роботи із різними структурами даних: чергами, деками, пріоритетними чергами, хеш-таблицями та стеками.

ЗМІСТ

ВСТУП 7
1 ПРЕДМЕТНА ОБЛАСТЬ ВИЗНАЧЕНОГО ПРОГРАМНОГО ПРОДУКТУ 9
1.1 Предметна область гри «Сокобан» 9
1.2 Постановка задачі. Загальний алгоритм 11
1.3 Аналіз аналогів та створення прототипу 12
1.4 Висновки до розділу 1 13
2 ТЕОРЕТИЧНІ ВІДОМОСТІ ЩОДО ФОРМУВАННЯ ТА ВИКОРИСТАННЯ БАЗОВИХ СТРУКТУР ДАНИХ ТА АЛГОРИТМІВ 15
2.1 Аналіз базових алгоритмів і структур даних 15
2.2 Особливості використання базових алгоритмів при створенні програми 16
2.3 Вибір мови програмування та особливості реалізації 18
2.4 Висновки до розділу 2 20
3 ПРОГРАМНА РЕАЛІЗАЦІЯ 21
3.1 Загальний алгоритм роботи програми 21
3.2 Реалізація базових алгоритмів 25
3.3 Тестування програми 26
3.4 Інструкція користувача 28
3.5 Висновки до розділу 3 29
ВИСНОВКИ 30
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ 32
Додаток А. Лістинг програми 34
Додаток Б. UML-діаграми програмної системи 37

ВСТУП

Гра «Сокобан» (Sokoban) [1] – це класична логічна головоломка, створена в Японії у 1981 році. На ігровому полі, що містить стіни та перешкоди, розташовані комірник, кілька ящиків і цільові комірки (склади). За один хід комірник може переміщуватися на сусідню вільну клітинку або штовхати перед собою рівно один ящик. Мета гри – розташувати всі ящики на цільових комірках за мінімальну кількість переміщень. Незважаючи на прості правила, задача є NP-складною [2], тому її розв'язання вимагає застосування ефективних алгоритмів пошуку в просторі станів, евристик та структур даних.

Актуальність теми курсової роботи зумовлена тим, що задачі пошуку шляху в просторі станів зустрічаються в багатьох практичних застосуваннях: планування маршрутів роботів, логістика складів, автоматизація ігор, планування дій у штучному інтелекті. Гра «Сокобан» є зручною модельною задачею для вивчення та порівняння алгоритмів пошуку, оскільки її простір станів добре формалізується, а складність зростає експоненційно з розміром поля та кількістю ящиків. Існуючі готові рішення (наприклад, онлайн-розв'язувачі Sokoban) часто не надають можливості порівняти різні алгоритми за часом, пам'яттю та якістю знайденого розв'язку, не показують процес пошуку покроково і не пояснюють внутрішні механізми роботи евристик.

Метою курсової роботи є програмна реалізація гри «Сокобан» та дослідження ефективності алгоритмів пошуку розв'язку в просторі станів: алгоритму пошуку в ширину (BFS) та евристичного алгоритму A* з різними оптимізаціями.

Об'єктом дослідження є програмний продукт – гра «Сокобан» з ручним та автоматичним режимами проходження. Предметом дослідження є алгоритми та структури даних, що використовуються для пошуку розв'язку: BFS, A*, виявлення тупикових станів, Угорський алгоритм, хешування Зобріста.

Для досягнення мети необхідно виконати такі завдання:

– проаналізувати предметну область використання програмного продукту та правила гри «Сокобан»;

– сформувати загальний алгоритм створюваного програмного продукту;

– проаналізувати базові алгоритми та структури даних, обрати ті, що будуть використані при створенні програмного продукту;

– обґрунтовано вибрати середовище та мову програмування;

– розробити щонайменше два алгоритми розв’язку завдання (BFS та A*) та оцінити їх часову та просторову складність;

– виконати програмну реалізацію з використанням об’єктно-орієнтованого підходу та щонайменше двох структур даних;

– перевірити роботу основних функцій програмного продукту, провести тестування та порівняльний аналіз алгоритмів.

У роботі використовуються методи аналізу та синтезу, алгоритмізації, моделювання, програмної реалізації та експериментального дослідження.

Структура курсової роботи: у першому розділі розглянуто предметну область гри «Сокобан», виконано постановку задачі та аналіз аналогів; у другому розділі проаналізовано базові алгоритми та структури даних, деталізовано алгоритми BFS та A*, обґрунтовано вибір мови програмування; у третьому розділі описано програмну реалізацію, тестування та інструкцію користувача. У висновках підбито підсумки виконаної роботи.

Прикладне значення програмного продукту: розроблена програма може використовуватися як навчальний інструмент для вивчення алгоритмів пошуку в просторі станів, для дослідження впливу евристик на ефективність пошуку, а також як основа для подальших досліджень у галузі планування дій та штучного інтелекту.

1 ПРЕДМЕТНА ОБЛАСТЬ ВИЗНАЧЕНОГО ПРОГРАМНОГО ПРОДУКТУ

1.1 Предметна область гри «Сокобан»

Гра «Сокобан» належить до класу логічних головоломок із повною інформацією. Відповідаючи на питання «Що?», «Для кого?» та «Навіщо?», можна сформулювати сутність програмного продукту.

Що? Програмний продукт являє собою гру «Сокобан» з двома режимами роботи: ручним проходженням рівнів користувачем та автоматичним пошуком розв'язку за допомогою дванадцяти алгоритмів: BFS, двох режимів A*, IDA*, Greedy, навчального X→Y, популяційних ACO, GA, Drain і Cannibal GA, біо-агента FlyBrain та зовнішнього ШІ. Програма завантажує рівні у стандартному текстовому форматі XSB, відображає ігрове поле, підтримує скасування ходів (Undo/Redo), анімацію знайденого розв'язку та виводить детальну статистику роботи алгоритмів: час пошуку, кількість досліджених станів, кількість ходів і штовхань із сортуванням рішень за часом, ходами та штовханнями.

Для кого? Продукт орієнтований на студентів, які вивчають алгоритми та структури даних, викладачів, а також усіх, хто цікавиться логічними іграми та штучним інтелектом. Завдяки наочній візуалізації процесу пошуку програма дозволяє побачити, як працюють алгоритми пошуку в просторі станів, чим відрізняється неінформований пошук від евристичного та як виявлення тупикових станів скорочує простір пошуку.

Навіщо? Необхідність програмного продукту зумовлена тим, що задача «Сокобан» є зручною модельною задачею для дослідження алгоритмів пошуку: її правила прості для розуміння, а простір станів достатньо складний, щоб продемонструвати переваги евристичного пошуку. Крім того, готові онлайн-розв'язувачі зазвичай не показують внутрішні механізми роботи алгоритмів, що ускладнює їх використання в навчальному процесі.

Формальна модель гри. Нехай поле має розмір M×N. Після вилучення стін залишається множина прохідних клітинок F. Множина цілей позначається G,

множина позицій ящиків – B, а позиція гравця – p. Кількість ящиків дорівнює кількості цілей: |B| = |G| = K. Повний стан гри задається як S = (p, B). Стіни та цілі не входять у динамічний стан, оскільки вони не змінюються під час проходження рівня, і належать статичному об’єкту Board.

Рух без штовхання допустимий, якщо сусідня клітинка p + d належить множині прохідних клітинок F і не зайнята ящиком. Штовхання допустиме, якщо перед гравцем розташований ящик (p + d ∈ B), а клітинка за ящиком p + 2d є вільною (належить F і не зайнята іншим ящиком). Тягнути ящик, пересувати два ящики одночасно або штовхати ящик у стіну заборонено. Рівень завершено, якщо кожен ящик стоїть на цілі: B = G.

Загальна архітектура програмної системи представлена на рисунку 1.1. Спільне ядро на мові C++20 реалізує правила гри та алгоритми пошуку, а два інтерфейси (термінальний CLI та Web) лише передають команди ядру та відображають отриманий стан. Такий підхід забезпечує однакову поведінку гри на всіх платформах і дозволяє тестувати алгоритми без запуску графічного інтерфейсу.

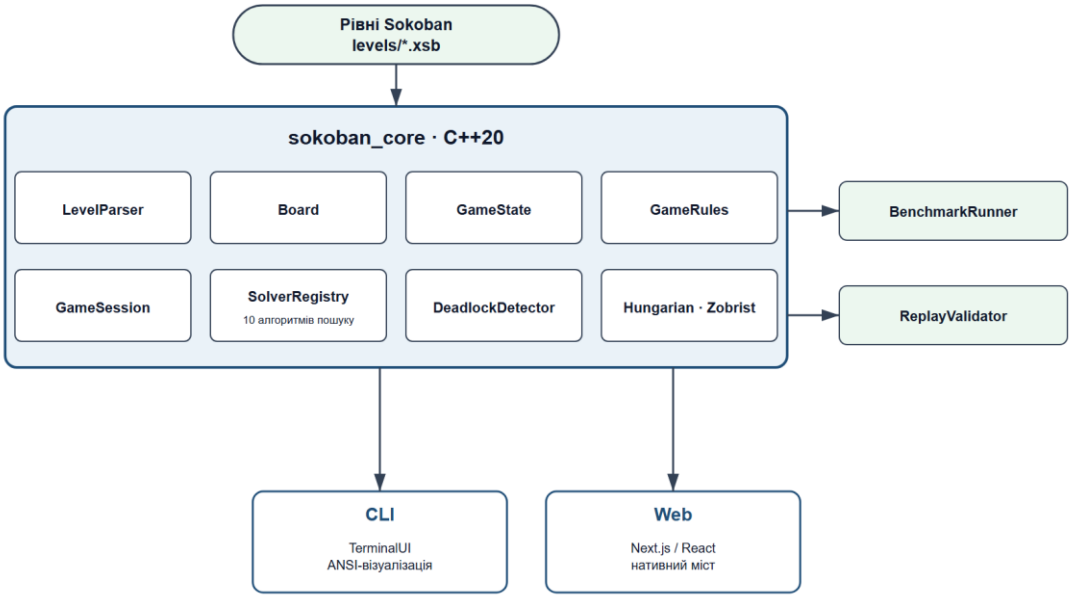


Рисунок 1.1 – Загальна архітектура програмної системи

Висновок до пункту: предметна область гри «Сокобан» добре формалізується, що дозволяє чітко визначити правила, стан гри та критерій завершення. Програмний продукт має чітке призначення – навчальне дослідження алгоритмів пошуку в просторі станів.

1.2 Постановка задачі. Загальний алгоритм

Відповідно до індивідуального завдання №74 необхідно розробити програму, яка реалізує гру «Сокобан» та автоматичний пошук її розв’язку. Постановка задачі включає такі вимоги:

– поле довільного розміру M×N зі стінами, підлогою та цілями;

– один гравець, K ящиків і K цілей;

– пересування гравця в чотирьох напрямках, штовхання лише одного ящика за один хід;

– ручний режим гри з підтримкою Undo/Redo та перезапуску рівня;

– автоматичне розв’язання рівня щонайменше двома алгоритмами: BFS та A*;

– пошук розв’язку з мінімальною кількістю ходів або штовхань;

– розпізнавання статичних і динамічних тупикових станів (Deadlock Detection);

– покрокове та автоматичне відтворення знайденого розв’язку;

– завантаження рівнів із текстових файлів стандартного формату Sokoban/XSB;

– виведення часу пошуку, кількості досліджених станів, ходів і штовхань для кожного алгоритму;

– використання об’єктно-орієнтованого підходу та щонайменше двох структур даних.

Загальний алгоритм роботи програми можна представити такою композицією кроків:

1) завантаження рівня з файлу XSB та його валідація (перевірка наявності рівно одного гравця, однакової кількості ящиків і цілей, відсутності невідомих символів);

2) побудова статичних даних поля: маски стін, цілей, списку прохідних клітинок, сусідів кожної клітинки, статичних мертвих клітинок та зворотних push-відстаней від клітинок до цілей;

3) вибір режиму роботи: ручна гра або автоматичний пошук;

4) у ручному режимі – обробка команд користувача через єдиний механізм GameSession::apply(), що перевіряє допустимість ходу за правилами гри;

5) в автоматичному режимі – запуск обраного алгоритму (BFS, A*, IDA*, Greedy, прототипів, популяційних методів, FlyBrain або зовнішнього ШІ) з поточного стану, збір статистики та відновлення послідовності ходів;

6) валідація знайденого розв’язку повторним прогоном через правила гри (ReplayValidator);

7) відображення результату: анімація ходів, кількість кроків і штовхань, час пошуку, кількість досліджених станів.

Поведінкові сценарії роботи програми описані в третьому розділі (режими Manual, Hint, AI Step, Autoplay, Compare). Висновок до пункту: постановка задачі є чіткою та структурованою, загальний алгоритм визначає послідовність обробки даних від завантаження рівня до відображення результату.

1.3 Аналіз аналогів та створення прототипу

На ринку існує значна кількість програмних продуктів, пов’язаних із грою «Сокобан». Розглянемо основні категорії аналогів.

Онлайн-розв’язувачі Sokoban (наприклад, sokoban-solver на GitHub, веб-сервіси типу sokobanonline.com) дозволяють завантажити рівень і отримати послідовність ходів. Переваги: швидке отримання результату, підтримка великих рівнів. Недоліки: відсутність пояснення процесу пошуку, неможливість порівняти алгоритми, закритий код, відсутність навчальної складової.

Ігрові реалізації «Сокобан» (мобільні додатки, браузерні ігри) зосереджені на ручному проходженні рівнів. Переваги: якісна графіка, зручне керування. Недоліки: відсутність автоматичного розв’язувача, статистики алгоритмів та навчальних матеріалів.

Навчальні демонстрації алгоритмів пошуку (наприклад, візуалізації BFS/A* на лабіринтах) показують процес пошуку, але зазвичай не використовують реальну механіку «Сокобан» зі штовханням ящиків і не демонструють специфічні для гри оптимізації: виявлення тупиків, макроштовхання, Угорський алгоритм.

Порівняльний аналіз аналогів наведено в таблиці 1.1.

Таблиця 1.1 – Порівняння програмних продуктів-аналогів

Критерій	**Онлайн-розв’язувачі**	**Ігрові реалізації**	**Навчальні демо**	**Розроблений продукт**
Ручний режим гри	–	+	–	+
Автоматичний пошук	+	–	+	+
Порівняння алгоритмів	–	–	частково	+
Статистика пошуку	частково	–	частково	+
Виявлення тупиків	+	–	–	+
Навчальна складова	–	–	+	+
Відкритий код	частково	–	+	+

Як прототип було створено мінімальну термінальну версію гри: завантаження рівня, рух гравця, штовхання ящика, перевірка перемоги. Прототип підтвердив коректність формальної моделі та дозволив перейти до реалізації алгоритмів пошуку. Висновок до пункту: жоден з розглянутих аналогів не поєднує ручну гру, автоматичний пошук двома алгоритмами, детальну статистику та навчальну візуалізацію, що підтверджує доцільність розробки власного програмного продукту.

1.4 Висновки до розділу 1

У першому розділі виконано аналіз предметної області гри «Сокобан»: визначено сутність, цільову аудиторію та призначення програмного продукту, формалізовано правила гри та стан системи. Сформульовано постановку задачі у

вигляді чіткого структурованого списку вимог та загальний алгоритм роботи програми. Проведено аналіз аналогів, який показав відсутність готових рішень, що поєднують усі необхідні функції, та створено прототип термінальної версії гри. Результати аналізу є основою для вибору алгоритмів і структур даних у другому розділі.

2 ТЕОРЕТИЧНІ ВІДОМОСТІ ЩОДО ФОРМУВАННЯ ТА ВИКОРИСТАННЯ БАЗОВИХ СТРУКТУР ДАНИХ ТА АЛГОРИТМІВ

2.1 Аналіз базових алгоритмів і структур даних

Алгоритм – це будь-яка коректно визначена обчислювальна процедура, на вхід якої подається певна величина чи набір величин, а результатом виконання є вихідна величина чи набір значень [3]. Ефективність алгоритму оцінюється за часом виконання та обсягом використаної пам’яті. Для оцінювання складності використовується асимптотична нотація «велике O»: O(1) – константна, O(log n) – логарифмічна, O(n) – лінійна, O(n log n) – квазілінійна, $O(n^2)$ – квадратична, $O(2^n)$ – експоненційна складність [4].

Структура даних – це спосіб організації та зберігання даних, що забезпечує ефективний доступ і модифікацію. Розглянемо базові структури даних, які використовуються в програмі [5]:

– масив (array) – послідовність елементів одного типу з доступом за індексом за O(1); використовується для зберігання масок стін, цілей, сусідів клітинок;

– черга (queue) – структура FIFO (перший прийшов – перший вийшов); операції додавання та вилучення виконуються за O(1); використовується в BFS для зберігання відкритих вузлів;

– дек (deque) – двостороння черга, що дозволяє додавати та вилучати елементи з обох кінців за O(1); використовується як черга BFS у реалізації std::deque;

– пріоритетна черга (priority queue) – структура, в якій елемент із найвищим пріоритетом вилучається першим; операції вставки та вилучення виконуються за O(log n); використовується в A* для зберігання відкритої множини;

– хеш-таблиця (hash table) – структура, що забезпечує доступ до елементів за ключем у середньому за O(1); використовується для Closed Set (множини відвіданих станів) та кешу евристики;

– стек (stack) – структура LIFO (останній прийшов – перший вийшов); використовується для реалізації Undo/Redo історії ходів;

– граф (graph) – математична модель, що складається з вершин і ребер; простір станів гри «Сокобан» природно представляється графом, де вершина – допустимий стан, а ребро – хід або штовхання.

Порівняння часової складності основних операцій структур даних наведено в таблиці 2.1.

Таблиця 2.1 – Часова складність основних операцій структур даних

Структура даних	**Доступ**	**Пошук**	**Вставка**	**Вилучення**
Масив	O(1)	O(n)	O(n)	O(n)
Черга / дек	O(n)	O(n)	O(1)	O(1)
Пріоритетна черга	O(1) (макс.)	O(n)	O(log n)	O(log n)
Хеш-таблиця	O(1) сер.	O(1) сер.	O(1) сер.	O(1) сер.
Стек	O(n)	O(n)	O(1)	O(1)

Висновок до пункту: для реалізації алгоритмів пошуку в просторі станів доцільно використовувати комбінацію структур даних: чергу/дек для BFS, пріоритетну чергу для A*, хеш-таблицю для Closed Set, стек для історії ходів та масиви для статичних даних поля.

2.2 Особливості використання базових алгоритмів при створенні програми

Простір станів гри «Сокобан» представляється графом, у якому кожна вершина – допустимий стан $S = (p, B)$, а ребро – команда руху або штовхання. Для F прохідних клитинок і K ящиків груба верхня межа кількості станів дорівнює $C(F, K) \times (F - K)$, що показує комбінаторний вибух навіть для невеликих полів. Саме тому потрібні компактне представлення стану, хешування, евристика та виявлення тупиків.

Існують два корисні види графа: граф окремих рухів, у якому кожне ребро має вартість 1, та граф штовхань, у якому ребро містить найкоротший шлях гравця до ящика та одне штовхання. Перший простіший і природний для BFS, другий значно менший і практичніший для A*.

Алгоритм BFS (пошук у ширину). BFS є повним неінформованим алгоритмом, який досліджує стани в порядку збільшення відстані від початкового. Якщо кожне ребро відповідає одному руху, BFS гарантує розв'язок із мінімальною кількістю рухів. Основні структури даних: std::deque для черги відкритих вузлів, std::unordered_map для множини відвіданих станів, вектор вузлів із батьківськими посиланнями.

Часова складність BFS становить O(V + E), де V – кількість досяжних станів, E – кількість переходів; просторова складність – O(V). Блок-схему алгоритму BFS наведено на рисунку 3.1.

Алгоритм A* (евристичний пошук). A* є інформованим алгоритмом, який для кожного вузла n обчислює оцінку f(n) = g(n) + h(n), де g(n) – точна накопичена вартість шляху від початкового стану до n, а h(n) – допустима нижня оцінка залишкової вартості від n до цілі. A* повертає оптимальний результат за обраною метрикою, якщо евристика ніколи не перевищує справжню мінімальну залишкову вартість (допустимість), усі вартості ребер невід'ємні, а стан повторно відкривається при знаходженні меншого g. У реалізації замість окремих кроків стану генеруються макроштовхання, що стискають десятки звичайних кроків в один макроперехід; повна послідовність UDLR зберігається для відтворення.

$$f(n) = g(n) + h(n) \tag{2.1}$$

де f(n) – оцінка повної вартості шляху через вузол n;; g(n) – фактична вартість уже виконаних ходів або штовхань від початкового стану до n;; h(n) – евристична нижня оцінка вартості переміщення ящиків на цілі від стану n.

Сильніша евристика використовує зворотні push-відстані: для кожної цілі виконується зворотний пошук положення умовного ящика, а отримана відстань є нижньою оцінкою реальної кількості штовхань.

Для врахування взаємного розташування ящиків формується матриця вартостей cost[i][j], де елемент – зворотна push-відстань від ящика i до цілі j, і знаходиться мінімальне взаємно однозначне призначення ящиків цілям за допомогою Угорського алгоритму зі складністю $O(K^3)$:

$$h(S) = min \Sigma cost[box_i][assigned_goal_i] \quad (2.2)$$

де h(S) – евристична оцінка стану S;; cost[box_i][assigned_goal_i] – зворотна push-відстань від ящика i до призначеної йому цілі;; K – кількість ящиків (і цілей).

Ця евристика сильніша за незалежний вибір найближчої цілі, оскільки кожна ціль призначається лише одному ящику. Якщо повне скінченне призначення не існує, конфігурація ящиків відкидається. Значення евристики кешується за канонічною конфігурацією ящиків.

Виявлення тупикових станів (Deadlock Detection). Тупиковий детектор має право відкинути лише стан, для якого доведено неможливість розв’язання; хибнопозитивний результат робить алгоритм неповним. Реалізовано перевірки: ящик у нецільовому куті, статичні мертві клітинки (зворотний push-пошук від усіх цілей), блок 2×2, заморожений ящик і неможливе призначення ящиків цілям.

Розташування ящиків змінюється лише після штовхання, тому динамічний Deadlock Detection запускається тільки після штовхання.

Хешування станів. Для Closed Set використовується хеш-таблиця з хешуванням Зобріста: для кожної пари «клітинка – тип об’єкта» за фіксованим seed генерується 64-бітне число, а хеш стану утворюється XOR-комбінацією відповідних чисел. Хеш-таблиця додатково порівнює повний канонічний стан.

Висновок до пункту: для розв’язання гри «Сокобан» обрано комбінацію алгоритмів BFS, A* та IDA* з макроштовханнями, зворотними push-відстанями, Угорським алгоритмом, безпечним Deadlock Detection та хешуванням Зобріста, що забезпечує повноту пошуку, оптимальність за обраною метрикою та високу практичну швидкодію.

2.3 Вибір мови програмування та особливості реалізації

Для реалізації програмного продукту розглянуто кілька мов програмування. Порівняння наведено в таблиці 2.2.

Таблиця 2.2 – Порівняння мов програмування для реалізації

Критерій	**C++20**	**Java**	**Python**	**C#**
Продуктивність пошуку	висока	середня	низька	середня
Контроль пам’яті	повний	частковий	відсутній	частковий
Стандартні структури даних	STL	Java Collections	вбудовані	.NET
Компіляція у WebAssembly	так	ні	частково	ні
Складність розробки	середня	середня	низька	середня
Підходить для ігор	так	частково	ні	частково

Мова C++20 була обрана з таких причин: висока продуктивність, необхідна для пошуку в просторі станів із мільйонами вузлів; багата стандартна бібліотека STL, що надає всі необхідні структури даних (deque, priority_queue, unordered_map, vector); можливість використання ядра як нативного процесу-моста для браузерної версії; підтримка об’єктно-орієнтованого програмування. Аргумент досвіду програмування також відіграє роль: C++ вивчається в межах дисциплін «Алгоритми та структури даних» та «Об’єктно-орієнтоване програмування».

Система збирання – CMake, що забезпечує кросплатформену збірку. Термінальний інтерфейс реалізовано з використанням ANSI escape-послідовностей для кольорів та переміщення курсора; у Windows вмикається підтримка Virtual Terminal та кодова сторінка UTF-8. Браузерна версія реалізована на Next.js/React: пошук виконує нативний процес-міст sokoban_web (те саме C++-ядро, ізольований процес на запит, JSON через stdin/stdout), а біо-агент FlyBrain працює у Web Worker, щоб сторінка не зависала. Для тестування використано власний легкий фреймворк модульних тестів.

Особливості реалізації: єдиний механізм команд GameSession::apply() для людини та ШІ гарантує, що автоматично знайдений шлях завжди перевіряється тими самими правилами, що й рухи користувача. Пошук виконується асинхронно з можливістю скасування; для вимірювання часу використовується монотонний

годинник std::chrono::steady_clock. Чистий час пошуку (search_time) вимірюється без завантаження рівня, малювання та анімації.

Висновок до пункту: мова C++20 та середовище збирання CMake є обґрунтованим вибором для реалізації високопродуктивного ядра гри «Сокобан» з можливістю повторного використання в різних інтерфейсах.

2.4 Висновки до розділу 2

У другому розділі проаналізовано базові алгоритми та структури даних, розглянуто алгоритми BFS та A*, що використовуються для пошуку розв’язку гри «Сокобан». Описано евристику на основі зворотних push-відстань із застосуванням Угорського алгоритму, виявлення тупикових станів та хешування Зобріста. Обґрунтовано вибір мови програмування C++20 та системи збирання CMake. Отримані теоретичні відомості є основою для програмної реалізації, описаної в третьому розділі.

3 ПРОГРАМНА РЕАЛІЗАЦІЯ

3.1 Загальний алгоритм роботи програми

Програмна система складається зі спільного C++20-ядра та двох інтерфейсів: термінального CLI і браузерної Web-версії. Ядро містить класи Board (статичне поле), GameState (динамічний стан), GameRules (правила гри), GameSession (керування грою, історія, Undo/Redo), LevelParser (завантаження XSB-рівнів), BFSSolver, AStarSolver, IDAStarSolver, GreedySolver, прототипи та популяційні розв’язувачі (реєстр SolverRegistry, загалом десять локальних алгоритмів), DeadlockDetector (виявлення тупиків), Hungarian (Угорський алгоритм), Zobrist (хешування) та ReplayValidator (перевірка розв’язків).

Ключовим архітектурним рішенням є єдиний механізм команд: і ручний гравець, і автоматичний розв’язувач надсилають однакові команди до одного об’єкта GameSession::apply(). Це гарантує, що автоматично знайдений шлях завжди перевіряється тими самими правилами, що й рухи користувача. Інтерфейс не змінює координати самостійно – він лише викликає apply(), отримує результат і показує його.

Стан гри представляється структурою GameState, що зберігає позицію гравця та відсортований вектор позицій ящиків. Позиції ящиків зберігаються у канонічному порядку, тому два стани з однаковим набором позицій ящиків і однаковою позицією гравця вважаються рівними незалежно від умовних номерів ящиків. Двовимірна координата (x, y) перетворюється на один індекс index = y × width + x, що спрощує масиви, хешування та порівняння станів.

Загальний алгоритм автоматичного розв’язання рівня:

1) завантаження та валідація рівня LevelParser::parseFile();

2) побудова статичних даних Board::finalize(): сусіди клітинок, статичні мертві клітинки, зворотні push-відстані;

3) створення розв’язувача через SolverRegistry::create(kind) та запуск start(board, state, options);

4) циклічний виклик advance(nodeBudget) до завершення пошуку;

5) відновлення послідовності ходів та перевірка ReplayValidator::validate();

6) виведення статистики: час пошуку, кількість досліджених станів, ходів, штовхань.

Блок-схему алгоритму BFS наведено на рисунку 3.1.

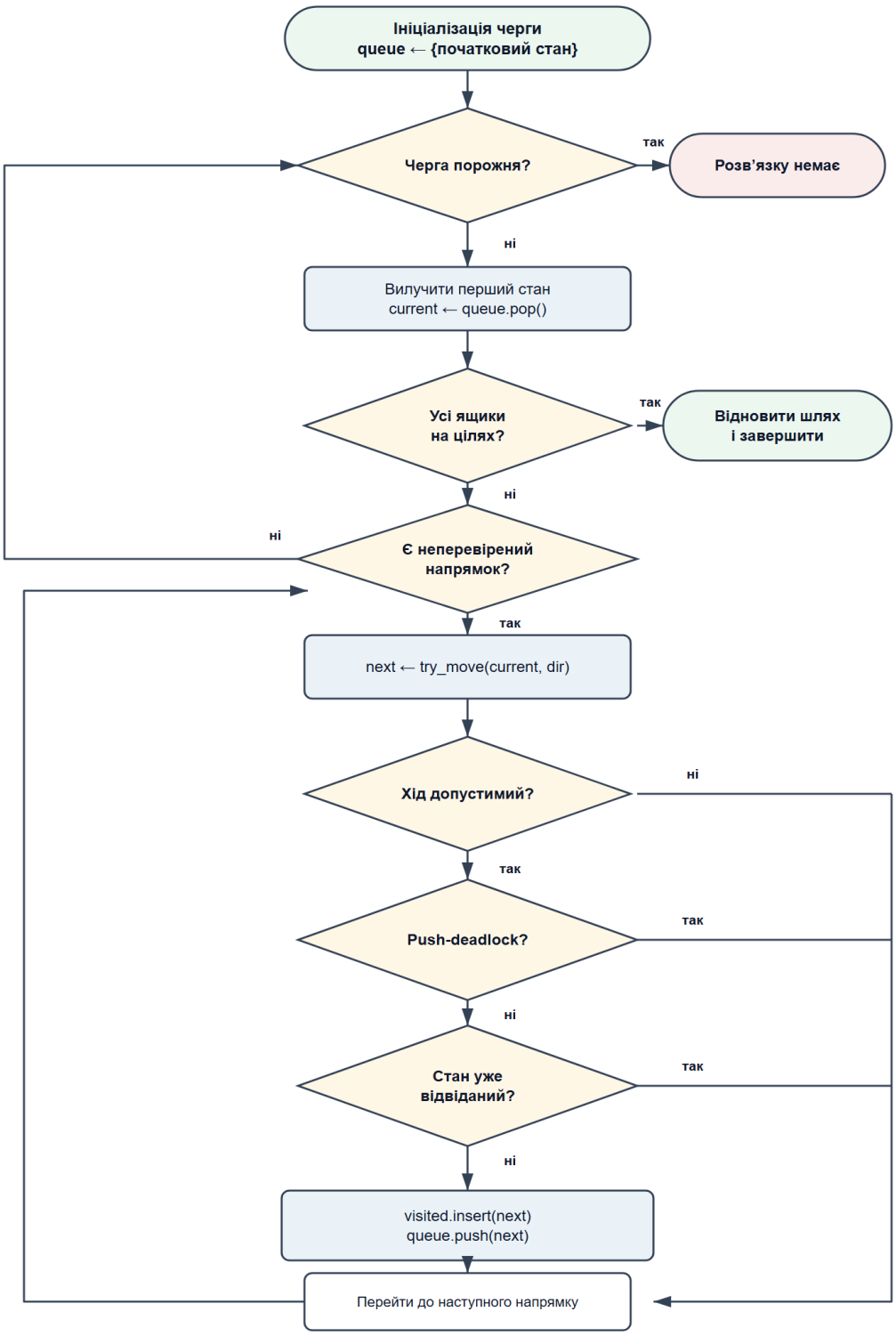


Рисунок 3.1 – Блок-схема алгоритму BFS

Блок-схему алгоритму A* наведено на рисунку 3.2.

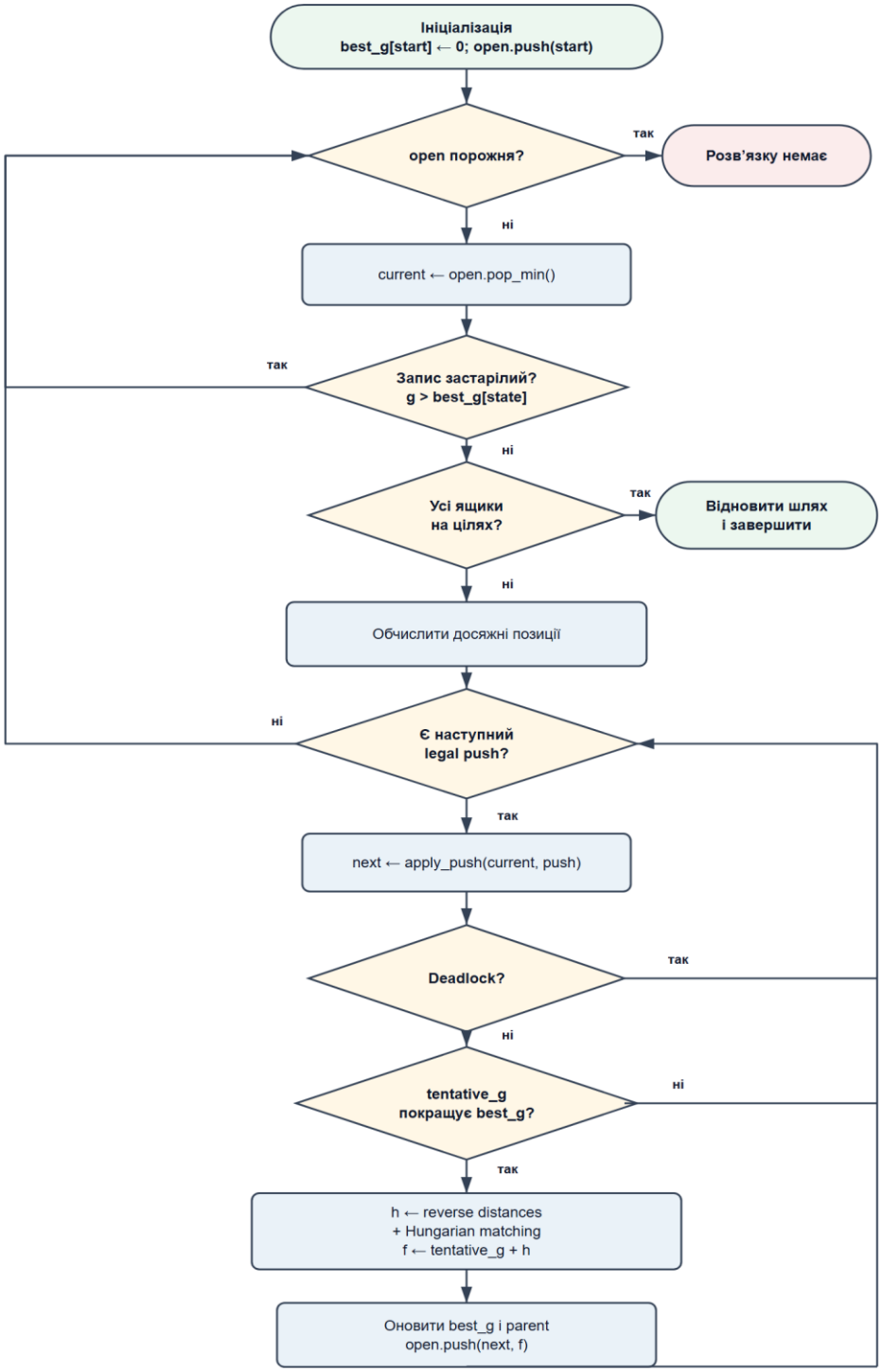


Рисунок 3.2 – Блок-схема алгоритму A*

Висновок до пункту: загальний алгоритм роботи програми реалізує повний цикл від завантаження рівня до відображення статистики, а єдиний механізм команд забезпечує коректність і узгодженість ручного та автоматичного режимів.

3.2 Реалізація базових алгоритмів

Реалізація BFS. Клас BFSSolver реалізує інтерфейс ISolver. Внутрішня структура BFSNode зберігає стан, індекс батьківського вузла, напрямок переходу, глибину та ознаку штовхання. Зберігання індексів замість вказівників спрощує серіалізацію і запобігає проблемам після розширення вектора. Черга відкритих вузлів реалізована на std::deque, множина відвіданих станів – на std::unordered_map з хеш-функцією StateHasher. Фіксований порядок напрямків (Up, Left, Down, Right) робить результат детермінованим: на однаковому рівні програма обирає однаковий шлях серед декількох однаково коротких.

Фрагмент реалізації BFS:

std::deque<std::uint32_t> queue_; std::vector<BFSNode> nodes_; std::unordered_map<GameState, std::uint32_t, StateHasher> visited_; // у циклі: current = queue_.front(); queue_.pop_front(); для кожного напрямку генерується next = try_move(...); якщо штовхання створило тупик (DeadlockDetector::isDeadlock) – стан відкидається; якщо стан уже відвіданий – пропускається; інакше зберігається parent і стан додається в чергу.

Реалізація A*. Клас AStarSolver використовує пріоритетну чергу std::priority_queue<OpenEntry> для відкритої множини, хеш-таблицю bestG_ для найкращої відомої вартості стану та кеш евристики heuristicCache_ за конфігурацією ящиків. Оскільки стандартна priority_queue не підтримує операцію decrease-key, допускається додавання нового запису з кращим g; під час вилучення застарілий запис розпізнається порівнянням із поточним найкращим g і пропускається (stale_pruned).

Перед генерацією штовхань виконується внутрішній BFS доступності гравця computeReachability(), який визначає маску досяжних клітинок, найкоротші відстані та батьківські напрямки для відновлення маршруту до позиції штовхання. Для кожного ящика і напрямку перевіряється допустимість макроштовхання, після чого обчислюється вартість ребра за обраною метрикою (Moves або Pushes)

та евристика computeHeuristic() з використанням Угорського алгоритму Hungarian::solve().

Фрагмент реалізації A*:

std::priority_queue<OpenEntry, std::vector<OpenEntry>, std::greater<OpenEntry>> openQueue_; std::unordered_map<GameState, std::uint64_t, StateHasher> bestG_; std::unordered_map<std::vector<CellIndex>, std::uint64_t, BoxConfigHasher> heuristicCache_; // OpenEntry порівнюється за f, потім за h, потім за порядковим номером вставки (детермінований tie-breaker).

Виявлення тупиків. Клас DeadlockDetector надає статичні методи isDeadlock(), hasCornerDeadlock(), has2x2Deadlock(), hasFreezeDeadlock(), hasAssignmentDeadlock(). Статичні перевірки (мертві клітинки) обчислюються один раз під час завантаження рівня, динамічні – лише після штовхання. Усі перевірки працюють у безпечному режимі (Safe mode), що гарантує відсутність хибнопозитивних спрацювань.

Хешування Зобріста. Клас Zobrist генерує 64-бітні випадкові числа для кожної пари «клітинка – тип об’єкта» за фіксованим seed, що забезпечує відтворюваність тестів. Хеш стану обчислюється XOR-комбінацією і може оновлюватися інкрементально після руху: movePlayer(hash, from, to) = hash XOR tablePlayer[from] XOR tablePlayer[to].

Висновок до пункту: програмна реалізація базових алгоритмів виконана з використанням об’єктно-орієнтованого підходу та необхідних структур даних: черги, деки, пріоритетної черги, хеш-таблиць, стеків і масивів. Реєстр SolverRegistry надає десять локальних алгоритмів, а веб-інтерфейс доповнює їх біо-агентом FlyBrain і зовнішнім ШІ.

3.3 Тестування програми

Для перевірки правильності реалізації використано власний фреймворк модульних тестів. Тестовий набір ядра містить 79 тестів, що охоплюють: правила гри (рух, штовхання, заборона штовхати два ящики, перемога), парсер XSB

(валідація рівнів, помилки формату), Undo/Redo, алгоритми BFS та A* (оптимальність, ліміти, нерозв'язні рівні), виявлення тупиків (кути, блоки 2×2, мертві клітинки, призначення), генератор рівнів та порівняння алгоритмів. Усі 79 тестів проходять успішно (79 Passed, 0 Failed); додатково 24 тести веб-моста (native.test.mjs, flybrain.test.mjs) перевіряють розв'язання всіх рівнів, повтори, генератор і біо-агента.

Функціональне тестування виконано на п'яти рівнях різної складності: 01_simple (1 ящик), 02_microban (1 ящик), 03_two_boxes (2 ящики), 04_warehouse (2 ящики), 05_triple (3 ящики). Третій слот інтерфейсу використовує файл 03_two_boxes.xsb; файл 03_microban_3.xsb навмисно нерозв'язний і використовується в негативному тесті. Для кожного рівня запускалися BFS (метрика Moves), A* (Moves) та A* (Pushes). Результати порівняння наведено в таблиці 3.1.

Таблиця 3.1 – Результати тестування алгоритмів на контрольних рівнях

Рівень	**Алгоритм**	**Метрика**	**Ходів**	**Штовхань**	**Досліджено станів**	**Відкинуто тупиків**	**Search, мс**
01_simple	BFS	Moves	1	1	2	0	0.003
01_simple	A*	Moves	1	1	2	2	0.008
01_simple	A*	Pushes	1	1	2	2	0.007
02_microban	BFS	Moves	7	1	18	3	0.012
02_microban	A*	Moves	7	1	2	3	0.007
02_microban	A*	Pushes	7	1	2	3	0.006
03_two_boxes	BFS	Moves	9	2	73	20	0.046
03_two_boxes	A*	Moves	9	2	6	16	0.033
03_two_boxes	A*	Pushes	9	2	3	8	0.027
04_warehouse	BFS	Moves	7	2	35	5	0.019
04_warehouse	A*	Moves	7	2	4	6	0.029
04_warehouse	A*	Pushes	7	2	3	4	0.021
05_triple	BFS	Moves	13	6	1198	166	0.735
05_triple	A*	Moves	13	6	31	53	0.273
05_triple	A*	Pushes	17	6	7	6	0.136

Аналіз результатів показує: BFS гарантує мінімальну кількість ходів, але досліджує значно більше станів (на рівні 05_triple – 1198 проти 31 у A* Moves). A* з евристикою та Deadlock Detection досліджує на 1–2 порядки менше станів, що підтверджує ефективність евристичного пошуку. Усі знайдені розв'язки

пройшли перевірку ReplayValidator (статус Passed), тобто повторний прогон через правила гри підтвердив їх коректність.

Окремо перевірено вплив оптимізацій A*: виявлення тупиків відкидає значну частину станів (наприклад, на рівні 05_triple – 53 стани для A* Moves), що скорочує час пошуку. Хешування Зобріста прискорює роботу Closed Set, а кешування евристики за конфігурацією ящиків усуває повторні обчислення Угорського алгоритму.

Висновок до пункту: тестування підтвердило коректність реалізації правил гри та алгоритмів пошуку, оптимальність знайдених розв’язків за обраною метрикою та ефективність використаних оптимізацій.

3.4 Інструкція користувача

Запуск термінальної версії: sokoban_cli.exe (головне меню), sokoban_cli.exe play <рівень.xsb> (гра), sokoban_cli.exe solve <рівень.xsb> --algorithm astar --metric pushes (автоматичне розв’язання), sokoban_cli.exe compare <рівень.xsb> (порівняння алгоритмів).

Керування в інтерактивній грі: WASD або стрілки – рух гравця та штовхання ящиків; U – скасування ходу (Undo); Y – повторення ходу (Redo); R – перезапуск рівня; H – підказка ШІ (перший хід A*); B / M / P – розв’язати алгоритмами BFS / A* Moves / A* Pushes; I / O / X – IDA*, Greedy та прототип X→Y; G – зовнішній ШІ; K – введення API-ключа; T – порівняти всі алгоритми з поточної позиції; пробіл – пауза/продовження відтворення; N – один крок розв’язку; Q або Esc – вихід.

Браузерна версія запускається командою npm run dev у каталозі web та відкривається за адресою http://localhost:3000. Вона надає вибір п’яти рівнів (Рівень 1 – Рівень 5) і генератора власних карт (/levels/create: поле 5–100 клітинок, 1–50 ящиків, точна кількість стін, seed), ручну гру з клавіатурою, камеру для полів понад 22 клітинки, покрокове відтворення рішень, сторінку налаштування

ключа ШІ (/ai, Groq або Google) та інтерактивні навчальні візуалізації алгоритмів (/about).

Список алгоритмів в інтерфейсі пронумеровано від 1 до 12 (BFS, два режими A*, IDA*, Greedy, X→Y, ACO, GA, Drain, Cannibal GA, FlyBrain, зовнішній ШІ) без дублювання метрики в назві; картки рішень показують час, досліджені стани, ходи та штовхання й сортуються за часом, ходами або штовханнями. Доступні порівняння всіх алгоритмів із поточної позиції та вибіркове порівняння поруч із синхронним відтворенням.

Висновок до пункту: інтерфейс програми є зручним для роботи, підтримує візуалізацію, покрокове відтворення та скасування ходів відповідно до вимог індивідуального завдання.

3.5 Висновки до розділу 3

У третьому розділі описано програмну реалізацію гри «Сокобан»: загальний алгоритм роботи, реалізацію базових алгоритмів BFS та A* з використанням об’єктно-орієнтованого підходу та структур даних, результати тестування та інструкцію користувача. Експериментальне порівняння підтвердило, що A* з евристикою та виявленням тупиків значно ефективніший за BFS на рівнях середньої складності, зберігаючи оптимальність за обраною метрикою.

ВИСНОВКИ

У ході виконання курсової роботи було розроблено програмний продукт – гру «Сокобан» з ручним та автоматичним режимами проходження, що відповідає індивідуальному завданню №74.

Виконано аналіз предметної області гри «Сокобан», формалізовано правила гри та стан системи, сформульовано постановку задачі у вигляді структурованого списку вимог та загальний алгоритм роботи програми. Проведено аналіз аналогів, який показав відсутність готових рішень, що поєднують ручну гру, автоматичний пошук дванадцятьма алгоритмами, детальну статистику та навчальну візуалізацію.

Розроблено базові алгоритми розв'язання задачі – пошук у ширину (BFS), що гарантує мінімальну кількість ходів на невеликих рівнях, та евристичний A* з макроштовханнями, зворотними push-відстанями, мінімальним призначенням ящиків цілям за допомогою Угорського алгоритму та безпечним виявленням тупикових станів. Оцінено часову та просторову складність алгоритмів: BFS має складність O(V + E) за часом і O(V) за пам'яттю, A* у найгіршому випадку також експоненційний, але завдяки евристиці та відсіканню тупиків досліджує на 1–2 порядки менше станів. До реєстру також входять IDA* (оптимальний, мало пам'яті), жадібний пошук, прототип X→Y, популяційні ACO, GA, Drain і Cannibal GA, а веб-інтерфейс надає біо-агента FlyBrain і зовнішній ШІ.

Програмну реалізацію виконано на мові C++20 з використанням об'єктно-орієнтованого підходу. У програмі використано щонайменше дві структури даних з переліку завдання: черги та деки (BFS), пріоритетні черги (A*), хеш-таблиці (Closed Set, кеш евристики), стеки (Undo/Redo) та масиви (статичні дані поля). Програма обчислює та виводить на екран час роботи кожного алгоритму, кількість досліджених станів, ходів і штовхань, має зручний інтерфейс із підтримкою візуалізації, покрокового відтворення та скасування ходів.

Тестування підтвердило коректність реалізації: усі 79 модульних тестів ядра проходять успішно (додатково 24 тести веб-моста), знайдені розв'язки проходять

повторну перевірку ReplayValidator. Експериментальне порівняння на контрольних рівнях показало, що A* досліджує значно менше станів, ніж BFS, зберігаючи оптимальність за обраною метрикою.

У ході розробки курсової роботи було вивчено базові алгоритми та структури даних, методи оцінювання складності алгоритмів, евристичний пошук, виявлення тупикових станів, хешування та об’єктно-орієнтоване програмування на C++. Отримано практичні навички роботи зі стандартною бібліотекою STL, системою збирання CMake та тестуванням програмного забезпечення.

Результати курсової роботи можуть використовуватися в навчальному процесі для вивчення алгоритмів пошуку в просторі станів, а також як основа для подальших досліджень: pattern database, навчувані евристики та розширення набору рівнів.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Sokoban [Електронний ресурс] // Wikipedia. – Режим доступу: https://en.wikipedia.org/wiki/Sokoban (дата звернення: 10.09.2025).

2. Culberson J. Sokoban is PSPACE-complete / J. Culberson // Proceedings of the International Conference on Fun with Algorithms. – 1999. – P. 65–76.

3. Кормен Т. Алгоритми: побудова та аналіз / Т. Кормен, Ч. Лейзерсон, Р. Рівест, К. Стайн. – 3-тє вид. – Київ: К.І.С., 2019. – 1296 с.

4. Коваленко О.О. Алгоритми та структури даних: навчальний посібник [Електронний ресурс] / О.О. Коваленко, О.М. Ткаченко, Р.Ю. Чехместрук. – Вінниця: ВНТУ, 2024. – 113 с.

5. Седжвік Р. Алгоритми на C++ / Р. Седжвік. – Київ: ДіаСофт, 2002. – 688 с.

6. Russell S. Artificial Intelligence: A Modern Approach / S. Russell, P. Norvig. – 4th ed. – Pearson, 2021. – 1115 p.

7. Hart P.E. A Formal Basis for the Heuristic Determination of Minimum Cost Paths / P.E. Hart, N.J. Nilsson, B. Raphael // IEEE Transactions on Systems Science and Cybernetics. – 1968. – Vol. 4, No. 2. – P. 100–107.

8. Kuhn H.W. The Hungarian Method for the Assignment Problem / H.W. Kuhn // Naval Research Logistics Quarterly. – 1955. – Vol. 2. – P. 83–97.

9. Zobrist A.L. A New Hashing Method with Application for Game Playing / A.L. Zobrist // ICCA Journal. – 1990. – Vol. 13, No. 2. – P. 69–73.

10. Junghanns A. Sokoban: Enhancing General Single-Agent Search Methods Using Domain Knowledge / A. Junghanns, J. Schaeffer // Artificial Intelligence. – 2001. – Vol. 129, No. 1–2. – P. 219–251.

11. Методичні вказівки до самостійної роботи студентів з виконання курсових робіт з дисципліни «Алгоритми та структури даних» / уклад.: О.О. Коваленко, О.М. Рейда, Л.М. Круподьорова. – Вінниця: ВНТУ, 2018. – 44 с.

12. C++ Reference [Електронний ресурс] // cppreference.com. – Режим доступу: https://en.cppreference.com (дата звернення: 10.09.2025).

13. CMake Documentation [Електронний ресурс] // cmake.org. – Режим доступу: https://cmake.org/documentation (дата звернення: 10.09.2025).

14. Sokoban Solver [Електронний ресурс] // GitHub. – Режим доступу: https://github.com/search?q=sokoban+solver (дата звернення: 10.09.2025).

Додаток А

Лістинг програми

Нижче наведено основні фрагменти програмного коду ядра гри «Сокобан» на мові C++20. Повний вихідний код проекту знаходиться у каталозі SAKOBAN-main.

A.1 Типи та напрямки (src/core/Types.hpp)

```
namespace sokoban::core {
using CellIndex = std::uint32_t;
constexpr CellIndex INVALID_CELL = std::numeric_limits<CellIndex>::max();


enum class Direction : std::uint8_t {
    Up = 0, Left = 1, Down = 2, Right = 3
};


constexpr char directionToChar(Direction dir) noexcept {
    switch (dir) {
        case Direction::Up: return 'U';
        case Direction::Left: return 'L';
        case Direction::Down: return 'D';
        case Direction::Right: return 'R';
    }
    return '?';
}


struct MoveRecord {
    CellIndex playerFrom = INVALID_CELL;
    CellIndex playerTo = INVALID_CELL;
    CellIndex boxFrom = INVALID_CELL;
    CellIndex boxTo = INVALID_CELL;
    CommandSource source = CommandSource::Human;
    Direction direction = Direction::Up;
    [[nodiscard]] bool pushedBox() const noexcept {
        return boxFrom != INVALID_CELL && boxTo != INVALID_CELL;
    }
};
} // namespace sokoban::core
```

A.2 Стан гри та хешування (src/core/GameState.hpp)

```
class GameState {
public:
    CellIndex player = INVALID_CELL;
    std::vector<CellIndex> boxes;


    [[nodiscard]] bool hasBox(CellIndex c) const noexcept {
        return std::binary_search(boxes.begin(), boxes.end(), c);
    }
    [[nodiscard]] bool isGoal(const Board& board) const noexcept {
        for (CellIndex b : boxes)
            if (!board.isGoal(b)) return false;
        return true;
    }
    bool operator==(const GameState& other) const noexcept {
        return player == other.player && boxes == other.boxes;
    }
};
```

```
struct StateHasher {
    std::size_t operator()(const GameState& s) const noexcept {
        std::uint64_t h = 0xcbf29ce484222325ULL; // FNV-1a
        auto mix = [&h](std::uint64_t v) { h ^= v; h *= 0x100000001b3ULL; };
        mix(static_cast<std::uint64_t>(s.player));
        for (CellIndex b : s.boxes) mix(static_cast<std::uint64_t>(b));
        return static_cast<std::size_t>(h);
    }
};
```

A.3 Інтерфейс розв'язувачів (src/solvers/ISolver.hpp)

```
class ISolver {
public:
    virtual ~ISolver() = default;
    virtual void start(const core::Board& board,
                       const core::GameState& state,
                       const SolverOptions& options) = 0;
    virtual SearchStatus advance(std::size_t nodeBudget) = 0;
    virtual void requestCancel() = 0;
    virtual SolverProgress progress() const = 0;
    virtual SearchStatistics statistics() const = 0;
    virtual std::optional<Solution> solution() const = 0;
};
```

A.4 Клас BFS-розв'язувача (src/solvers/BFSSolver.hpp)

```
class BFSSolver : public ISolver {
public:
    void start(const core::Board& board,
               const core::GameState& state,
               const SolverOptions& options) override;
    SearchStatus advance(std::size_t nodeBudget) override;
    void requestCancel() override;
    SearchStatistics statistics() const override;
    std::optional<Solution> solution() const override;
private:
    struct BFSNode {
        core::GameState state;
        std::uint32_t parentIndex = 0;
        core::Direction move = core::Direction::Up;
        std::uint32_t depth = 0;
        bool pushed = false;
    };
    std::deque<std::uint32_t> queue_;
    std::vector<BFSNode> nodes_;
    std::unordered_map<core::GameState, std::uint32_t,
                       core::StateHasher> visited_;
    std::atomic<bool> cancelRequested_{false};
};
```

A.5 Клас A*-розв'язувача (src/solvers/AStarSolver.hpp)

```
class AStarSolver : public ISolver {
public:
    void start(const core::Board& board,
               const core::GameState& state,
               const SolverOptions& options) override;
    SearchStatus advance(std::size_t nodeBudget) override;
    SearchStatistics statistics() const override;
    std::optional<Solution> solution() const override;
private:
    struct OpenEntry {
        std::uint64_t f = 0, g = 0, h = 0, seq = 0;
        std::uint32_t nodeIndex = 0;
        bool operator>(const OpenEntry& other) const noexcept {
            if (f != other.f) return f > other.f;
            if (h != other.h) return h > other.h;
            return seq > other.seq; // детермінований tie-breaker
        }
```

```
    };
    std::priority_queue<OpenEntry, std::vector<OpenEntry>,
                        std::greater<OpenEntry>> openQueue_;
    std::unordered_map<core::GameState, std::uint64_t,
                       core::StateHasher> bestG_;
    std::unordered_map<std::vector<core::CellIndex>, std::uint64_t,
                       core::BoxConfigHasher> heuristicCache_;
    std::unique_ptr<Zobrist> zobrist_;
};
```

A.6 Виявлення тупиків (src/solvers/DeadlockDetector.hpp)

```
class DeadlockDetector {
public:
    static bool isDeadlock(const core::Board& board,
                           const core::GameState& state,
                           core::CellIndex movedBox = core::INVALID_CELL,
                           bool checkAssignment = false);
    static bool hasStaticDeadSquare(const core::Board& board,
                                    const core::GameState& state) noexcept;
    static bool hasCornerDeadlock(const core::Board& board,
                                  const core::GameState& state,
                                  core::CellIndex movedBox) noexcept;
    static bool has2x2Deadlock(const core::Board& board,
                               const core::GameState& state,
                               core::CellIndex movedBox) noexcept;
    static bool hasFreezeDeadlock(const core::Board& board,
                                  const core::GameState& state) noexcept;
    static bool hasAssignmentDeadlock(const core::Board& board,
                                      const core::GameState& state);
};
```

A.7 Угорський алгоритм (src/solvers/Hungarian.hpp)

```
class Hungarian {
public:
    static constexpr std::int64_t INF_COST = 1'000'000'000LL;
    // Мінімальне дводольне призначення N рядків до M стовпців (N <= M)
    static std::pair<std::int64_t, std::vector<int>>
    solve(const std::vector<std::vector<std::int64_t>>& costMatrix);
};
```

A.8 Хешування Зобріста (src/solvers/Zobrist.hpp)

```
class Zobrist {
public:
    explicit Zobrist(std::size_t boardSize,
                     std::uint64_t seed = 0x504b42414e3230ULL);
    std::uint64_t computeHash(const core::GameState& state) const noexcept;
    std::uint64_t movePlayer(std::uint64_t hash, core::CellIndex from,
                             core::CellIndex to) const noexcept {
        return hash ^ tablePlayer_[from] ^ tablePlayer_[to];
    }
    std::uint64_t moveBox(std::uint64_t hash, core::CellIndex from,
                          core::CellIndex to) const noexcept {
        return hash ^ tableBox_[from] ^ tableBox_[to];
    }
private:
    std::vector<std::uint64_t> tablePlayer_;
    std::vector<std::uint64_t> tableBox_;
};
```

A.9 Приклад рівня у форматі XSB (levels/02_microban.xsb)

```
; Level 02 - Classic Microban
#######
#     #
# $ # #
# . # #
#    @#
#######
```

Додаток Б

UML-діаграми програмної системи

На рисунку Б.1 наведено діаграму класів програмної системи «Сокобан». Ядро містить класи Board, GameState, GameRules, GameSession та LevelParser; алгоритми пошуку реалізують спільний інтерфейс ISolver; допоміжні класи Hungarian, Zobrist та DeadlockDetector забезпечують евристику, хешування та відсікання тупиків. Реєстр SolverRegistry надає десять локальних розв'язувачів через спільний інтерфейс ISolver.

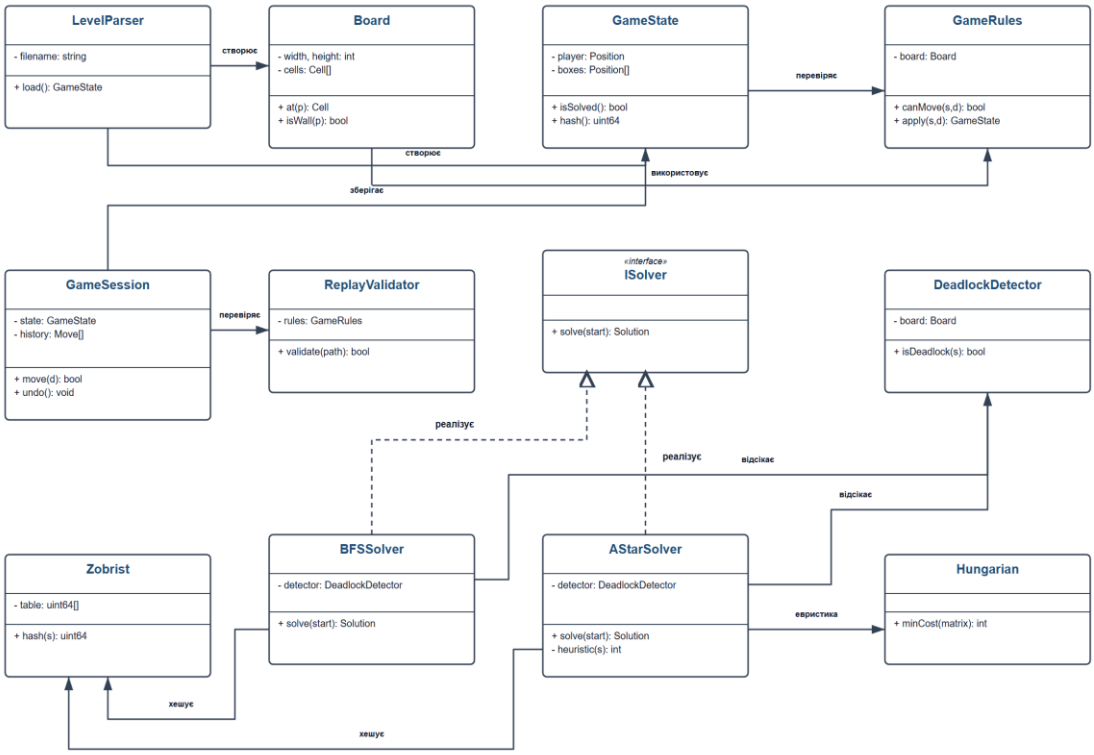


Рисунок Б.1 – Діаграма класів програмної системи «Сокобан»

На рисунку Б.2 наведено діаграму послідовності автоматичного розв'язання рівня: користувач ініціює пошук, CLI створює розв'язувач через реєстр, алгоритм виконує пошук із використанням статичних даних поля, виявлення тупиків та евристики, після чого знайдений розв'язок перевіряється ReplayValidator.

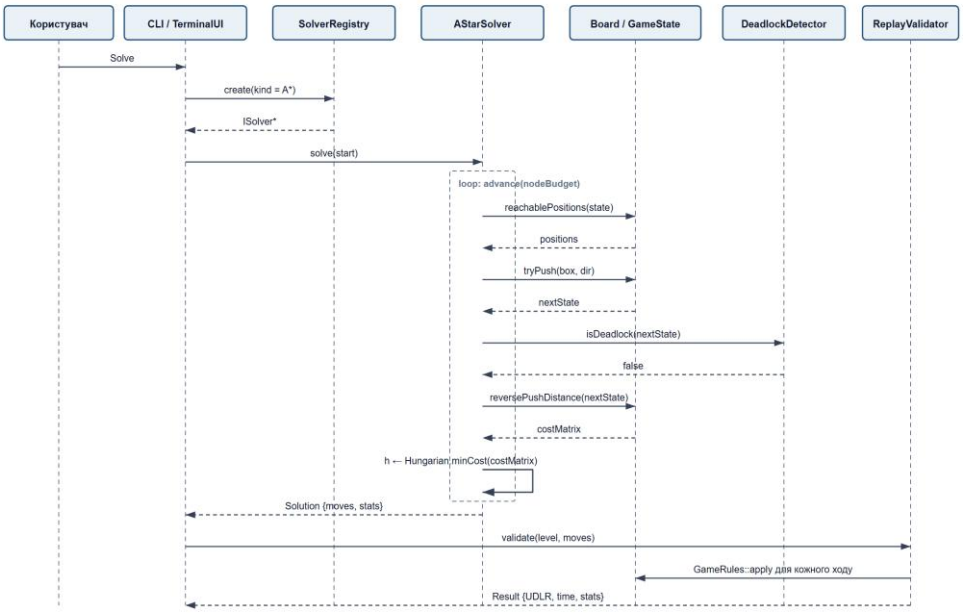


Рисунок Б.2 – Діаграма послідовності автоматичного розв’язання рівня

На рисунку Б.3 наведено діаграму станів режимів взаємодії користувача та ШІ.

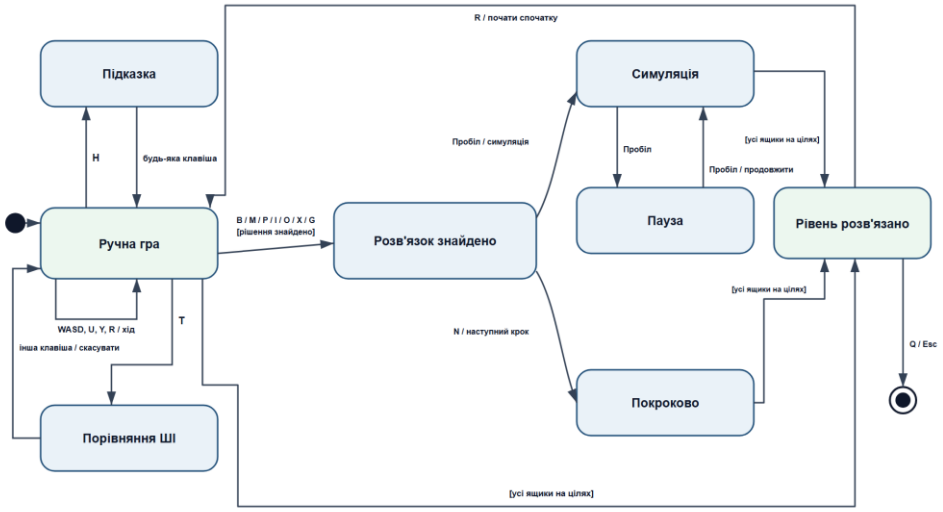


Рисунок Б.3 – Діаграма станів режимів взаємодії користувача та ШІ

Діаграма охоплює стани «Ручна гра», «Підказка», «Розв’язок знайдено», «Симуляція», «Пауза», «Покроково», «Порівняння ШІ» та «Рівень розв’язано», а також початковий і кінцевий стани.